



PYTHON NOTES — PYTHON DICTIONARIES AND TUPLES (CONTINUED)



TOPIC: TUPLES

After learning dictionaries, the next important Python data structure is the **tuple**.

A tuple is used to store multiple pieces of information together, just like a list. However, tuples have one very important difference:

A tuple cannot be changed after it has been created.

This property is called **immutability**.

Tuples are commonly used whenever information should remain fixed and should not be changed accidentally.

1. WHAT IS A TUPLE?

A tuple is an ordered collection of items.

Just like lists, tuples:

- Store multiple items
- Keep items in order
- Can contain different data types
- Can contain duplicate values
- Can be accessed using indexes

However, unlike lists:

- Tuples **cannot** be modified after creation.
-

2. CREATING A TUPLE

A tuple is created using **parentheses** `()`.

Example:

```
fruits = ('apple', 'banana', 'mango')
```

We can print the tuple:

```
print(fruits)
```

Output:

```
('apple', 'banana', 'mango')
```

3. ACCESSING TUPLE ITEMS

Tuple items are accessed using indexes.

Remember:

Python starts counting from **0**.

Example:

```
fruits = ('apple', 'banana', 'mango')  
  
print(fruits[0])
```

Output:

```
apple
```

Another example:

```
print(fruits[2])
```

Output:

```
mango
```

Index positions:

```
0 → apple  
1 → banana  
2 → mango
```

4. NEGATIVE INDEXING

Python can also count from the end.

Example:

```
fruits = ('apple', 'banana', 'mango')  
  
print(fruits[-1])
```

Output:

```
mango
```

Other examples:

```
print(fruits[-2])
```

Output:

```
banana
```

```
print(fruits[-3])
```

Output:

```
apple
```

Negative indexes count backwards.

5. SLICING TUPLES

Just like lists, tuples can be sliced.

Example:

```
fruits = ('apple', 'banana', 'mango', 'orange')  
  
print(fruits[1:3])
```

Output:

```
('banana', 'mango')
```

The last position is **not included**.

6. TUPLES ARE IMMUTABLE

The biggest characteristic of tuples is that they are **immutable**.

Immutable means:

They cannot be changed after they are created.

Example:

```
fruits = ('apple', 'banana', 'mango')  
  
fruits[0] = 'orange'
```

Python produces an error:

```
TypeError: 'tuple' object does not support item assignment
```

This is because tuples do not allow item assignment.

7. WHY ARE TUPLES IMMUTABLE?

Sometimes information should never change.

Examples include:

- Months of the year
- Days of the week
- RGB color values
- Coordinates
- Book genres
- Student ID numbers
- GPS positions

Making these values immutable helps prevent accidental changes.

8. LOOPING THROUGH A TUPLE

We can loop through tuples just like lists.

Example:

```
subjects = (  
    'Economics',  
    'Chemistry',  
    'Literature',  
    'English',  
    'Mathematics'  
)  
  
for subject in subjects:  
    print(subject)
```

Output:

```
Economics  
Chemistry  
Literature  
English  
Mathematics
```

This is one of the most common ways to use tuples.

9. USING `len()`

The `len()` function counts the number of items in a tuple.

Example:

```
subjects = (  
    'Economics',  
    'Chemistry',  
    'Literature',  
    'English'  
)  
  
print(len(subjects))
```

Output:

```
4
```

The tuple contains four items.

10. TUPLES CAN STORE DIFFERENT DATA TYPES

A tuple can contain different types of information.

Example:

```
profile = (  
    'Ben',  
    13,  
    'Maths',  
    True  
)  
  
print(profile)
```

Output:

```
('Ben', 13, 'Maths', True)
```

The tuple contains:

- String
- Integer
- String
- Boolean

Python allows different data types to exist inside the same tuple.

11. SINGLE-ITEM TUPLES

Creating a tuple with only one item is slightly different.

Incorrect:

```
color = ('blue')
```

Python treats this as a string.

Correct:

```
color = ('blue',)
```

Output:

```
('blue',)
```

The comma tells Python that this is a tuple.

12. TUPLE PACKING

Packing means placing several values into one tuple.

Example:

```
school_info = 'Chris', 'SS2', 14
```

Python automatically packs these values into a tuple.

Printing:

```
print(school_info)
```

Output:

```
('Chris', 'SS2', 14)
```

Notice that parentheses were optional.

Python added them automatically.

13. TUPLE UNPACKING

Unpacking means taking values out of a tuple and placing them into variables.

Example:

```
student = (  
    'Ben',  
    13,  
    'Maths'  
)  
  
name, age, subject = student
```

Now:

```
print(name)  
print(age)  
print(subject)
```

Output:


```
Ben  
13  
Maths
```

Python automatically matches:

```
Ben → name  
  
13 → age  
  
Maths → subject
```

The number of variables must match the number of items in the tuple.

14. TUPLE METHODS

Unlike lists, tuples have very few methods because tuples cannot be changed.

The two main methods are:

- `.count()`
- `.index()`

These methods do **not** modify the tuple.

They only help us retrieve information.

End of Part 1

In Part 2 we will cover:

- `.count()`
- `.index()`
- Tuple vs List
- Real-world uses of tuples
- Our Library Management System project
- Why we did not create a standalone tuples project
- Progress tracker
- Dictionaries and Tuples Complete